

AI Phone Assistant

A Rust backend that holds a phone conversation one sentence at a time. The phone side (or the built-in mimic client) turns the caller's speech into text and sends it to the backend. The backend sends back the sentence to say and what the phone should do next. Instead of one large prompt that tries to do everything, the work is split between small, single-purpose language-model *machines*, with plain Rust code in between them that owns every decision and all of the call's state.

1 Machines and their system prompts

A **machine** is one narrowly scoped call to the language model. It is defined by three things: a *role* (one or two sentences on who it is), a list of *rules*, and an *output format*. The output format is an ordinary Rust struct. Each of its fields describes the value it expects and, where it applies, the exact list of values it allows. The system prompt is generated from that struct. The same struct also produces a JSON schema that vLLM enforces while it generates the answer. So the model cannot answer with an unknown intent, invent an extra key, or return free text instead of JSON.

Every prompt has the same XML layout, so the model always finds things in the same place. The caller's sentence goes into the user message as `<utterance>`. Everything that comes from the caller or from a backend is escaped, so it can never close a tag and pass itself off as an instruction. The role and the rules are written as they are.

The prompt is deterministic. Context keys are sorted and the temperature is 0, so the same call state always produces the same prompt and the same answer. The values a turn adds, such as `backend_context`, go last and sit right next to the utterance, where the model pays them the most attention. Answers are capped at 256 tokens, which stops a runaway generation from blocking the GPU.

```
<role>You are the intent matcher of a phone...</role>
<rules>
  <rule>Set `caller_intent` to: ... Use only a
    value listed in <allowed_values></rule>
  <rule>A greeting or small talk is `unsupported`...
</rules>
<allowed_values> every intent label </allowed_values>
<json_output_format> field → JSON type
<conversation_history> earlier turns
<context>
  <current_time> <form_state>
  <language> <backend_context>
</context>
— user message —
<utterance>I'd like to see a doctor</utterance>
```

2 The machines in use, and the order they run in

Machine #1: intent matcher

Works out what the caller wants. It returns a one-line `machine_reasoning`, the detected `language` (ISO 639-1), a `caller_intent` from the fixed list, and the `form_field` and `form_field_value` when the sentence contains a form answer. A confidence score is read from the token probabilities of the answer. `INTENT_MATCHER` picks the implementation: `llm` (default; Qwen2.5-7B-Instruct on vLLM) or `classifier` (a GLiNER2 zero-shot model on the CPU, with the form logic written in Rust).

Machine #2: response formulator

Writes the sentence the caller hears: one or two sentences in plain text, in the caller's language, ready for text-to-speech. It may use only the facts in `backend_context`. Two variants handle special cases: the *form informant* answers questions about the form in progress from its state, and the *calendar refuser* politely turns down date arithmetic it could get wrong. A repeat skips machine #2 and returns the last answer word for word.

1 Session Load the call by <code>x-call-id</code> and take the per-call lock.	2 Machine #1 Find the intent, the language and the form value.	3 Fetch For information intents, call the external API.	4 Events Update the form and write the backend context.	5 Machine #2 Phrase the spoken answer from that context.	6 Save Store the turn and the state in Redis, and reply.
---	--	---	---	--	--

■ language model ■ plain Rust code, deterministic

The split is the core idea. The model answers two questions only: *what did the caller mean* and *how should the reply sound*. Everything in between is ordinary code. That code decides which form field to fill, whether a value still needs to be confirmed, and which question to ask next. It hands machine #2 a short instruction such as *"Recorded 'Anna Weber' for patient_name. Next, ask the caller to confirm that Full name of the patient is Anna Weber."* Each form field moves through three states, `queued` → `awaiting_confirmation` → `completed`, so every value is read back before it is accepted. After three rejected values for the same field, the assistant offers a human. When the last field is confirmed, a `FormCompleted` event sends the form as JSON to `FORM_SUBMIT_URL` in the background.

3 Mimic client and call sessions

The backend has no telephony of its own. The **mimic client** plays the phone's part, so the assistant can be tested by voice from a browser. It is a single HTML file with no dependencies, served by the app at `http://localhost:8080/mimic-client`. It uses Chrome's or Edge's built-in Web Speech API: *SpeechRecognition* turns the microphone into text (this step needs an internet connection), and *speechSynthesis* reads the answer aloud in the language the backend detected. Under each answer the page shows the matched function, the confidence, the language, the action and the reasoning, so you can see why the assistant answered the way it did.

How the call_id session works

- Every page load creates a new call id (`crypto.randomUUID()`) and sends it in the `x-call-id` header with each request. A request without it is rejected with HTTP 400.
- The id is the key of the call's session in Redis: the call state (idle, or a form in progress) and the conversation history that both machines see.
- A per-call lock runs the turns of one call one after another, so each turn sees what the previous one saved.
- The session is saved after every turn and expires one hour after the last one. Reloading the page starts a new call. A real phone integration sends the provider's call id instead.

How to use it

1. Start the stack with `docker compose up` (Postgres, Redis, vLLM, classifier, app).
2. Open `/mimic-client` in Chrome and press *Start call*. Choose English, Deutsch or Русский.
3. Speak, or type a sentence and press Enter. Typing goes through exactly the same endpoint.
4. When the action is `end_call` or `transfer_to_human`, the page hangs up.

Any HTTP client can do the same:

```
curl -X POST localhost:8080/assistant/handle-request \
-H "Content-Type: application/json" \
-H "x-call-id: test-call-1" \
-d '{"request_text": "Book a doctor appointment"}'
```

The reply carries `answer` (what the caller hears), `action` (what the phone does next), plus `selected_function`, `confidence`, `language_detected` and `reasoning`.

4 What it can do today, and how to extend it

Capability	What the caller gets
Live information	Current weather in Berlin (Open-Meteo) and the official EUR → UAH rate (National Bank of Ukraine).
Guided forms	Doctor appointment: patient name, date of birth, reason and preferred date, each read back for confirmation. The caller can correct any value, give a value for a later field early, or cancel.
Call control	Answer questions about the form from its state ("What name did you record?"), repeat the last answer, end the call, transfer to a human, politely refuse calendar questions.
Anything else	A polite "I can't help with that", followed by a list of what the assistant does offer. Every reply comes in the caller's language.

Most extensions are a new enum variant plus a few lines of code. The intent lists, the allowed values in the prompt, the JSON schema and the "what I can help with" list are all generated from the enums, so they cannot fall out of step with each other. A compile-time guard flags any intent that is added without being registered.

To add	Where	Good candidates
An information function	A variant of <code>GetInformationSupported</code> and a <code>fetch()</code> branch that calls any HTTP API or database.	Order or delivery status, opening hours, free appointment slots, account balance, ticket status, FAQ or knowledge-base lookup.
A form	A variant of <code>FormSupported</code> and its fields in <code>build()</code> . Field kinds: text, numbers, yes/no, date, spoken date.	Callback request, table or taxi booking, repair or service request, insurance claim intake, complaint, sales lead.
A reaction to an event	An <code>EventHandler</code> for <code>FormCompleted</code> , <code>IntentExtracted</code> or <code>CallSessionLoaded</code> .	SMS or e-mail confirmation, a CRM or calendar entry, saving to Postgres, and caller number or customer profile added to the context.
A phone action	A variant of <code>CallAction</code> , set by the event handler.	Send an SMS link, put the caller on hold, route to a specific department.
A reply style	A machine #2 variant with its own role and rules.	Strict legal or medical wording, a summary read back at the end of the call.